

Machine Learning-Based Detection of Phishing URLs Using Lexical URL Features

Saleh Alsaheb

*Networks and Information Security Engineering
Princess Sumaya University for Technology (PSUT)
Amman, Jordan*

Ismael Alhindi

*Networks and Information Security Engineering
Princess Sumaya University for Technology (PSUT)
Amman, Jordan*

Abstract—Phishing web pages remain a practical threat because they imitate legitimate sites and attempt to steal credentials, financial information, or other sensitive data. This project evaluates whether lightweight URL-based lexical features, augmented with six engineered ratio and binary indicators, can support reliable phishing detection with classical supervised machine learning methods. The experiments use the Kaggle Web Page Phishing Dataset, which contains 100,077 rows, 19 URL-derived numerical features, and one binary target label. After removing 78,186 duplicate rows, the modeling workflow is run on 21,891 unique records. Six additional features are derived from the original columns to capture symbol density, redirection presence, and URL depth. The pipeline includes dataset inspection, duplicate handling, feature engineering, stratified train/test splitting, feature scaling where required, five-fold cross-validation, hyperparameter tuning via GridSearchCV for all four models, and held-out test evaluation. The four evaluated models are Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. Gradient Boosting achieved the strongest held-out test performance with 0.8198 accuracy, 0.8505 precision, 0.9118 recall, 0.8801 F1-score, and 0.8873 ROC-AUC. All four models achieved F1-scores between 0.853 and 0.880, demonstrating that the combination of feature engineering and systematic hyperparameter tuning produces consistent and competitive results across the evaluated model set.

I. INTRODUCTION

Phishing attacks are a common cybersecurity problem because they trick users into interacting with fraudulent pages that appear trustworthy. Attackers often rely on deceptive URLs, domain patterns, and redirection behavior to mislead victims. Because phishing campaigns can change quickly and operate at scale, automated detection is an important defensive layer.

URL-based detection is attractive because it is lightweight and does not require downloading webpage content, executing scripts, or querying richer external sources such as WHOIS or DNS. Lexical properties such as URL length, the

number of dots, the number of hyphens, and the presence of special characters can provide useful signals about suspicious behavior. These properties are also straightforward to extract and evaluate in a batch machine learning workflow.

This project frames phishing detection as a supervised binary classification problem. The task is to classify a URL as legitimate or phishing, training and evaluation use a labeled public dataset, and the performance is measured using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix analysis.

Because a missed phishing page is more dangerous than a false alarm, recall is especially important.

The main contributions of this project are:

1. A reproducible Python implementation for phishing URL detection using lexical URL features.
2. A comparison of four classical and ensemble machine learning models under a shared evaluation workflow.
3. A structured analysis of class balance, feature relationships, confusion matrix behavior, and recall/F1 trade-offs.
4. A literature-based comparison with related phishing URL detection studies that use similar lexical features.

II. RELATED WORK

Blum et al. studied phishing URL detection using lexical features and online learning methods [2]. Their work demonstrated that useful phishing signals can be extracted directly from URL text without relying on full webpage content. The present project is related in its emphasis on lexical features, but differs in dataset choice and in the broader comparison of batch classifiers.

Sahingoz et al. examined machine learning based phishing detection from URLs using lexical and host-based signals [3]. In their reported experiments, Random Forest with

NLP-based features achieved 97.98% accuracy on a large URL dataset containing 36,400 legitimate and 37,175 phishing URLs. The present project is narrower because it focuses on the Kaggle Web Page Phishing Dataset and a simple reproducible pipeline centered on URL lexical features.

Awasthi and Goel evaluated both base and ensemble classifiers on two URL-feature phishing datasets using 30 structural URL features and 10-fold cross-validation [4]. Their base classifiers, including Logistic Regression, Decision Tree, and SVM, achieved accuracy in the range of 93 to 96 percent, which is methodologically relevant because it uses comparable classifier families and URL-based features, although the datasets and feature sets differ.

Alnemari and Alshammari evaluated Random Forest, Decision Tree, SVM, and ANN on the UCI phishing website dataset of 11,055 instances described by 30 URL-structural features [5]. Random Forest achieved the highest result at 97.3 percent accuracy, while SVM and Decision Tree fell in the 94 to 96 percent range. Their work is relevant because it uses the same classical model families as the current project on a comparable URL-feature set.

Although this work uses the Kaggle Web Page Phishing Dataset, the compared studies use different but related phishing URL datasets. The comparison therefore focuses on methodology, feature type, classifiers, and reported metric ranges rather than claiming a direct same-dataset benchmark.

III. EXPERIMENTAL SETUP

A. Dataset Description

The dataset used in this project is the Kaggle Web Page Phishing Dataset [1]. In the local CSV used for implementation, the dataset contains 100,077 rows and 20 columns before cleaning. Nineteen columns are numerical URL-derived features, and the target column is `phishing`, where 1 denotes phishing and 0 denotes legitimate. After duplicate removal, 21,891 unique rows remain for modeling. Six additional features are then engineered from the original columns, producing 25 features in total.

Table I lists the input features and the target variable used in the pipeline.

TABLE I
FEATURES USED IN THE PHISHING DETECTION PIPELINE

Feature	Type	Description
<code>url_length</code>	Numerical	Total characters in the URL
<code>n_dots</code>	Numerical	Count of <code>.</code> characters
<code>n_hyphens</code>	Numerical	Count of hyphen characters
<code>n_underline</code>	Numerical	Count of <code>_</code> characters
<code>n_slash</code>	Numerical	Count of <code>/</code> characters
<code>n_questionmark</code>	Numerical	Count of <code>?</code> characters
<code>n_equal</code>	Numerical	Count of <code>=</code> characters
<code>n_at</code>	Numerical	Count of <code>@</code> characters
<code>n_and</code>	Numerical	Count of <code>&</code> characters
<code>n_exclamation</code>	Numerical	Count of <code>!</code> characters
<code>n_space</code>	Numerical	Count of space characters
<code>n_tilde</code>	Numerical	Count of <code>~</code> characters
<code>n_comma</code>	Numerical	Count of <code>,</code> characters
<code>n_plus</code>	Numerical	Count of <code>+</code> characters
<code>n_asterisk</code>	Numerical	Count of <code>*</code> characters
<code>n_hashtag</code>	Numerical	Count of <code>#</code> characters
<code>n_dollar</code>	Numerical	Count of <code>\$</code> characters
<code>n_percent</code>	Numerical	Count of <code>%</code> characters
<code>n_redirection</code>	Numerical	Count of redirect-like markers
<code>total_special_chars</code>	Engineered	Sum of all special character counts
<code>symbol_ratio</code>	Engineered	$\text{total_special_chars} / (\text{url_length} + 1)$
<code>has_redirect</code>	Engineered	Binary: 1 if <code>n_redirection</code> > 0
<code>has_at</code>	Engineered	Binary: 1 if <code>n_at</code> > 0
<code>slash_dot_ratio</code>	Engineered	$\text{n_slash} / (\text{n_dots} + 1)$
<code>url_length_log</code>	Engineered	$\log(\text{url_length} + 1)$
<code>phishing</code>	Binary	1 = phishing, 0 = legitimate (target)

No missing values were observed in the cleaned dataset. After duplicate removal, the class distribution was 6,019 legitimate URLs and 15,872 phishing URLs, corresponding to roughly 27.5% legitimate and 72.5% phishing samples.



Fig. 1. Class distribution after deduplication (6,019 legitimate vs 15,872 phishing).

B. Exploratory Data Analysis

The exploratory analysis focuses on class balance, feature distributions, and feature-target relationships. The correlation analysis shows that `n_slash`, `url_length`, `n_equal`, `n_redirection`, and `n_dots` have the strongest absolute linear relationships with the phishing label in this dataset. Several count-based features exhibit noticeable skew and long tails, especially `url_length`, `n_hypens`, and `n_slash`.

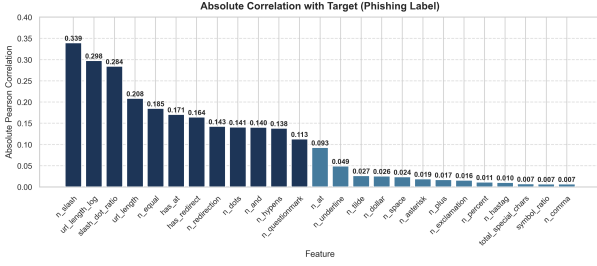


Fig. 2. Absolute Pearson correlation of features with the phishing label (ranked).

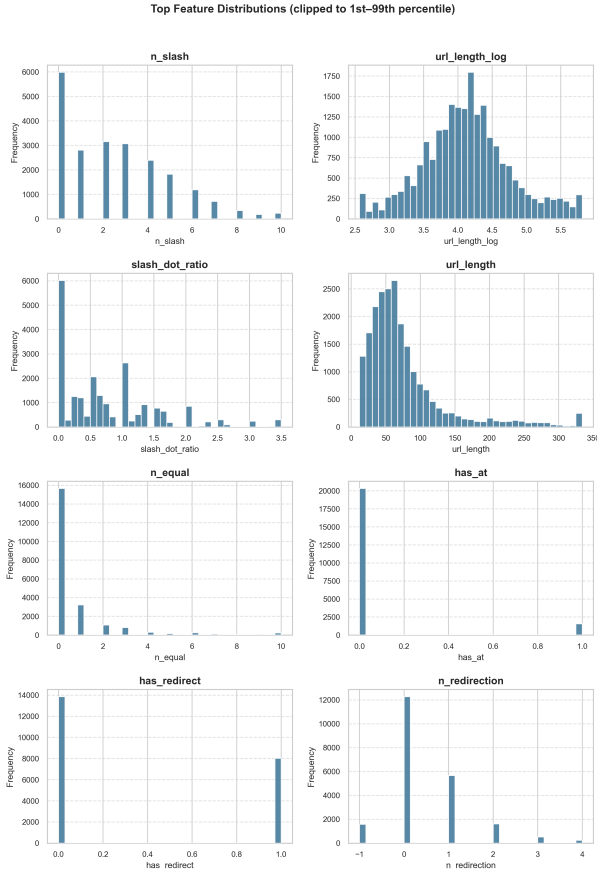


Fig. 3. Feature distributions clipped to the 1st-99th percentile range.

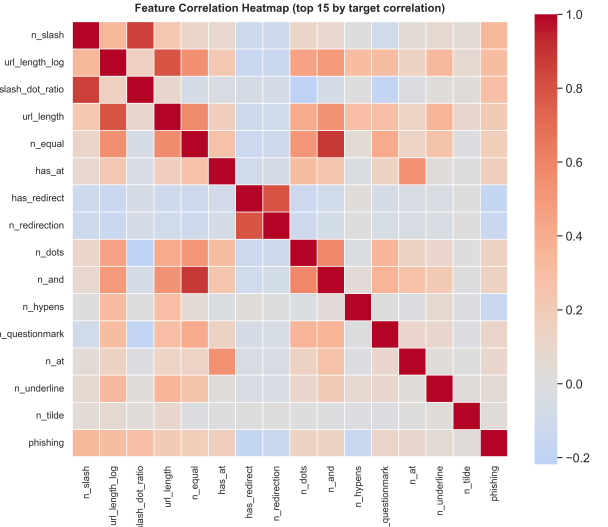


Fig. 4. Feature correlation heatmap. Warmer colors (red) indicate positive correlation; cooler (blue) indicate negative.

C. Data Preprocessing

Duplicate rows are removed once before model splitting, which is especially important in this dataset because more than three quarters of the original rows are duplicates. After deduplication, six features are engineered: a total special character count, a symbol-to-length ratio, binary redirect and at-symbol flags, a slash-to-dot ratio, and a log-transformed URL length. A stratified 80/20 train/test split is used to preserve class proportions. Feature scaling is applied only to Logistic Regression.

D. Evaluation Metrics

The project uses accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix analysis. Recall deserves special attention because false negatives correspond to phishing pages that evade detection, which carries the highest security risk.

IV. MACHINE LEARNING ALGORITHMS

Four classifiers are evaluated to cover the spectrum from simple linear models to complex boosting ensembles. Each model is tuned independently with GridSearchCV using 5-fold stratified cross-validation scored on F1.

A. Logistic Regression

Logistic Regression is included as the linear baseline. It models the log-odds of the phishing class as a weighted sum of the input features. Standard scaling is applied upstream in the pipeline. The regularization parameter C is tuned across 0.01, 0.1, 1, and 10.

B. Decision Tree

The Decision Tree classifier recursively partitions the feature space by selecting the split that maximizes information gain at each node. Maximum depth, minimum samples per split, and minimum samples per leaf are all tuned to control overfitting.

C. Random Forest

Random Forest is a bagging ensemble that trains a collection of decision trees on bootstrapped subsets of the training data with randomized feature subsets at each split. The number of estimators, maximum depth, and leaf size parameters are tuned via grid search.

D. Gradient Boosting

Gradient Boosting is a sequential boosting ensemble that builds trees one at a time, each correcting the residual errors of the previous ensemble. The learning rate, number of estimators, maximum tree depth, and subsampling ratio are tuned together to balance bias and variance.

V. RESULTS AND ANALYSIS

A. Model Performance

Table II summarizes the held-out test performance of all four models after hyperparameter tuning via GridSearchCV.

TABLE II

TEST PERFORMANCE AFTER HYPERPARAMETER TUNING

Model	Accuracy	Precision	Recall	F1	AUC
Gradient Boosting	0.8198	0.8505	0.9118	0.8801	0.8873
Random Forest	0.8102	0.8429	0.9074	0.8740	0.8793
Decision Tree	0.8022	0.8476	0.8866	0.8667	0.8478
Logistic Regression	0.7753	0.8101	0.9014	0.8533	0.8329

Gradient Boosting achieved the best overall result, delivering the highest accuracy, F1-score, and ROC-AUC. Random Forest was a close second with F1 = 0.874. Decision Tree improved substantially from an untuned F1 of 0.791 to 0.867 after hyperparameter tuning. The overall F1 range of 0.853 to 0.880 indicates consistent and competitive performance across the evaluated model set.

Since phishing samples form about 72.5% of the deduplicated dataset, accuracy alone is not sufficient; therefore, F1-score, recall, ROC-AUC, and confusion matrix behavior are emphasized throughout the evaluation.

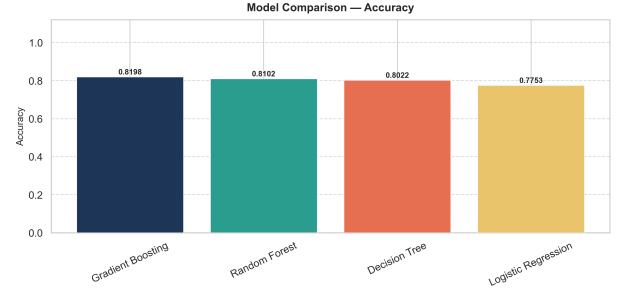


Fig. 5. Accuracy comparison across all four evaluated models.

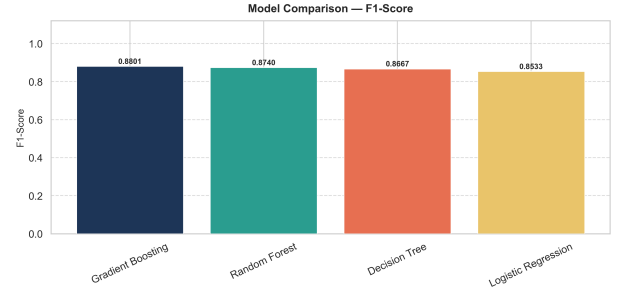


Fig. 6. F1-score comparison across all four evaluated models.

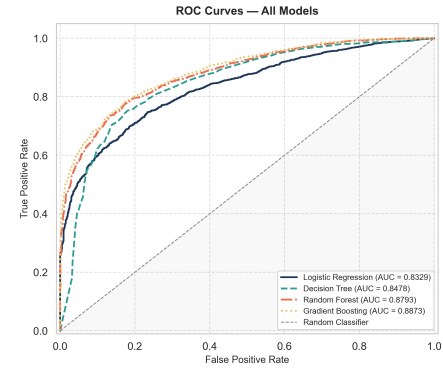


Fig. 7. ROC curves for all evaluated models. AUC values shown in legend.

B. Best Model Confusion Matrix

Table III shows the confusion matrix for Gradient Boosting on the held-out test set.

TABLE III

CONFUSION MATRIX OF THE GRADIENT BOOSTING MODEL

Actual / Predicted	Legitimate	Phishing
Legitimate	695	509
Phishing	280	2895

The model correctly identifies 2,895 of 3,175 phishing URLs, yielding recall = 0.912. The 509 false positives represent legitimate URLs flagged as phishing, a security-oriented trade-off, although the false-positive rate should be reduced before real-world deployment.

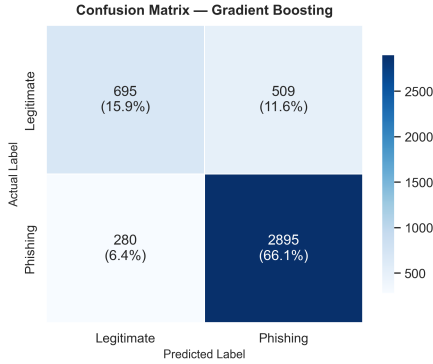


Fig. 8. Confusion matrix for Gradient Boosting on the held-out test set.

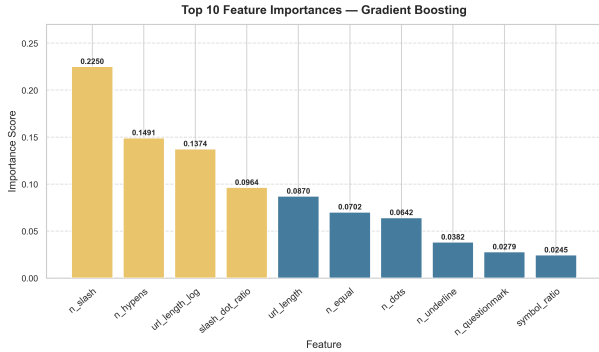


Fig. 9. Top feature importances for Gradient Boosting (best model).

C. Discussion

1. Gradient Boosting achieved $F1 = 0.8801$ and $ROC-AUC = 0.8873$, the best results overall.
2. Decision Tree improved substantially ($F1$: 0.791 to 0.867), demonstrating the importance of hyperparameter tuning.
3. All four models are competitive with $F1$ spanning 0.853-0.880.
4. Engineered features `symbol_ratio` and `slash_dot_ratio` appear in the top feature importances of tree-based models (Fig. 9).
5. Results differ from [4] and [5] in absolute accuracy due to stricter deduplication and a more limited feature set, but the evaluated model families are methodologically comparable.

VI. DISCUSSION AND LIMITATIONS

This project uses URL lexical features augmented with six engineered indicators, which makes the method lightweight and fast. The engineered features, specifically the symbol density ratio, slash-to-dot ratio, and binary redirect and at-symbol flags, provide additional discriminative signals that complement the raw counts and benefit linear models that cannot learn such relationships on their own.

Even with feature engineering, lexical features alone may miss sophisticated phishing pages that use cleaner-looking URLs or rely on infrastructure-level deception. The compared studies that report higher accuracy generally use richer feature sets that include structural, host-based, or content-based attributes. The current pipeline intentionally limits itself to URL-text features for reproducibility and computational simplicity.

Another limitation is dataset freshness. A model trained on a static public dataset may not fully reflect the newest phishing strategies. The current pipeline does not inspect webpage content, WHOIS data, DNS signals, SSL information, or dynamic behavior. Real-world deployment would require external validation on more recent data.

The duplicate structure of the provided CSV is a notable limitation. Removing duplicates reduced the working dataset from 100,077 rows to 21,891 unique rows. Results from experiments that do not deduplicate may appear artificially stronger.

VII. CONCLUSION

This project demonstrates a complete machine learning workflow for phishing URL detection using a public dataset of lexical URL features augmented with six engineered indicators. After deduplication, experiments were conducted on 21,891 unique URL records. All four models were tuned via GridSearchCV, and hyperparameter optimization produced a substantial improvement in Decision Tree performance ($F1$: 0.791 to 0.867) and modest but consistent gains across the ensemble models.

Among the evaluated models, Gradient Boosting achieved the strongest overall test performance with 0.8198 accuracy, 0.8505 precision, 0.9118 recall, 0.8801 $F1$ -score, and 0.8873 $ROC-AUC$. These results are methodologically aligned with previous studies [4], [5], but the absolute accuracy is lower, likely because this project applies strict duplicate removal and uses a limited lexical feature set.

URL lexical features combined with derived indicators can provide effective phishing detection without content-based inspection. Future work should validate the approach on newer datasets and expand the feature set with host, DNS, SSL, or content-based signals.

REFERENCES

- [1] Danielfernandon, "Web Page Phishing Dataset," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/danielfernandon/web-page-phishing-dataset>. Accessed: May 2026.

- [2] A. Blum, B. Wardman, T. Solorio, and G. Warner, "Lexical feature based phishing URL detection using online learning," in Proc. 3rd ACM Workshop on Artificial Intelligence and Security, 2010, pp. 54–60.
- [3] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Systems with Applications, vol. 117, pp. 345–357, 2019.
- [4] A. Awasthi and N. Goel, "Phishing website prediction using base and ensemble classifier techniques with cross-validation," Cybersecurity, vol. 5, no. 1, p. 22, 2022.
- [5] S. Alnemari and M. Alshammari, "Detecting phishing domains using machine learning," Applied Sciences, vol. 13, no. 8, p. 4649, 2023.